
GRAFIKA KOMPUTEROWA - LAB 10

Table of Contents

Joanna Dagil 231008	1
CEL CWICZENIA	1
ZADANIA	1
ZADANIE 1	2
ZADANIE 2	7
FUNKCJE POMOCNICZE	10
SHADOW	10
LAMBERT	13
PHONG	14
Funkcja do zamiany współrzędnych matematycznych na pikselowe i odwrotnie	15
Funkcja rysuje odcinek na obrazie monochromatycznym.	18
Funkcja do wyznaczania parametrów płaszczyzny $Ax+By+Cz+D=0$ przechodzącej przez trzy punkty $p1, p2, p3$...	20
Funkcja do wyznaczania odległości punktu $[x, y, z]$ od płaszczyzny o równaniu $Ax+By+Cz+D=0$	21
Funkcja wypełnia zamknięty obszar obrazu metoda floodfill.	21

Joanna Dagil 231008

clear; close all; clc;

CEL CWICZENIA

Rozszerzenie modeli oświetlenia i cieniowania - kontynuacja.

ZADANIA

% Dane są dwa trójkąty o następujących współrzędnych narożników:

% Trójkąt 1:

tri(1).A = [9; 3; 16];

tri(1).B = [-3; 9; 6];

tri(1).C = [-3; -3; 4];

% Trójkąt 2:

tri(2).A = [3; -1; 2];

tri(2).B = [-9; 6; 10];

tri(2).C = [-9; -9; 18];

% Współczynnik odbicia

e = 1;

% Scena ta wyświetlana jest na obrazie uzyskanym przez rzutowanie równoległe na płaszczyznę XY i zdyskretyzowanym do rozdzielczości $X \times Y = 256 \times 256$

X = 256;

Y = 256;

% przy czym rozmiar pojedynczego piksela to 0.1×0.1 .

```

dx = 0.1;
dy = 0.1;

% Oświetlenie tła to
amb = 50;

% Dodatkowo scena oświetlona jest pojedynczym punktowym źródłem światła
umieszczonym w
lampa = [-6; 0; -10];
lampb = [ 0; 0; -10];
% Natężenie źródła światła wynosi
int = 205;

```

ZADANIE 1

Używając modelu oświetlenia Lamberta, wygeneruj obraz tej sceny. Zwróć uwagę na to, że trójkąty mogą: - nawzajem się przesłaniać lub przecinać (a więc uwzględnij użycie algorytmu z-buforowania) - rzucać na siebie cień (w zależności od ich wzajemnego położenia względem źródła światła).

```

Im = uint8(zeros(Y,X));

[Im1, Ish1] = lambert(Im, tri(1), amb, lampa, int, X, Y, dx, dy);
[Im2, Ish2] = lambert(Im, tri(2), amb, lampa, int, X, Y, dx, dy);

figure(1); imshow(Im1);
figure(2); imshow(Im2);

% Cienie
[A1, B1, C1, D1] = plane(tri(1).A, tri(1).B, tri(1).C);
n1 = [A1; B1; C1] / norm([A1; B1; C1]);
if n1(3) > 0
    n1 = -1 * n1;
end

[A2, B2, C2, D2] = plane(tri(2).A, tri(2).B, tri(2).C);
n2 = [A2; B2; C2] / norm([A2; B2; C2]);
if n2(3) > 0
    n2 = -1 * n2;
end

Im1 = shadow(Im1, Ish1, Ish2, A1, B1, C1, D1, A2, B2, C2, D2, X, Y, dx, dy,
lampa, amb);
Im2 = shadow(Im2, Ish2, Ish1, A2, B2, C2, D2, A1, B1, C1, D1, X, Y, dx, dy,
lampa, amb);

figure(3); imshow(Im1);
figure(4); imshow(Im2);

% Z-buffer

Im = uint8(zeros(Y,X));
z_buffer = inf(Y,X);

```

```

for j = 1:Y
    for i = 1:X
        [xx,yy] = pix_to_mat(i, j, X, Y, dx, dy);
        if Ish1(j,i) == 255
            [d,~,~,~] = dist_to_plane([A1, B1, C1, D1], xx, yy, 0, 0, 0, 1);
            if d < z_buffer(j, i)
                z_buffer(j, i) = d;
                Im(j, i, :) = Im1(j, i, :);
            end
        end
        if Ish2(j,i) == 255
            [d,~,~,~] = dist_to_plane([A2, B2, C2, D2], xx, yy, 0, 0, 0, 1);
            if d < z_buffer(j, i)
                z_buffer(j, i) = d;
                Im(j, i, :) = Im2(j, i, :);
            end
        end
    end
end
end

```

```
figure(5); imshow(Im);
```

```
Im = uint8(zeros(Y,X));
```

```

[Im1, Ish1] = lambert(Im, tri(1), amb, lampb, int, X, Y, dx, dy);
[Im2, Ish2] = lambert(Im, tri(2), amb, lampb, int, X, Y, dx, dy);

```

```

figure(6); imshow(Im1);
figure(7); imshow(Im2);

```

```
% Cienie
```

```

[A1, B1, C1, D1] = plane(tri(1).A, tri(1).B, tri(1).C);
n1 = [A1; B1; C1] / norm([A1; B1; C1]);

```

```

if n1(3) > 0
    n1 = -1 * n1;
end

```

```

[A2, B2, C2, D2] = plane(tri(2).A, tri(2).B, tri(2).C);
n2 = [A2; B2; C2] / norm([A2; B2; C2]);

```

```

if n2(3) > 0
    n2 = -1 * n2;
end

```

```

Im1 = shadow(Im1, Ish1, Ish2, A1, B1, C1, D1, A2, B2, C2, D2, X, Y, dx, dy,
lampb, amb);

```

```

Im2 = shadow(Im2, Ish2, Ish1, A2, B2, C2, D2, A1, B1, C1, D1, X, Y, dx, dy,
lampb, amb);

```

```
figure(8); imshow(Im1);
```

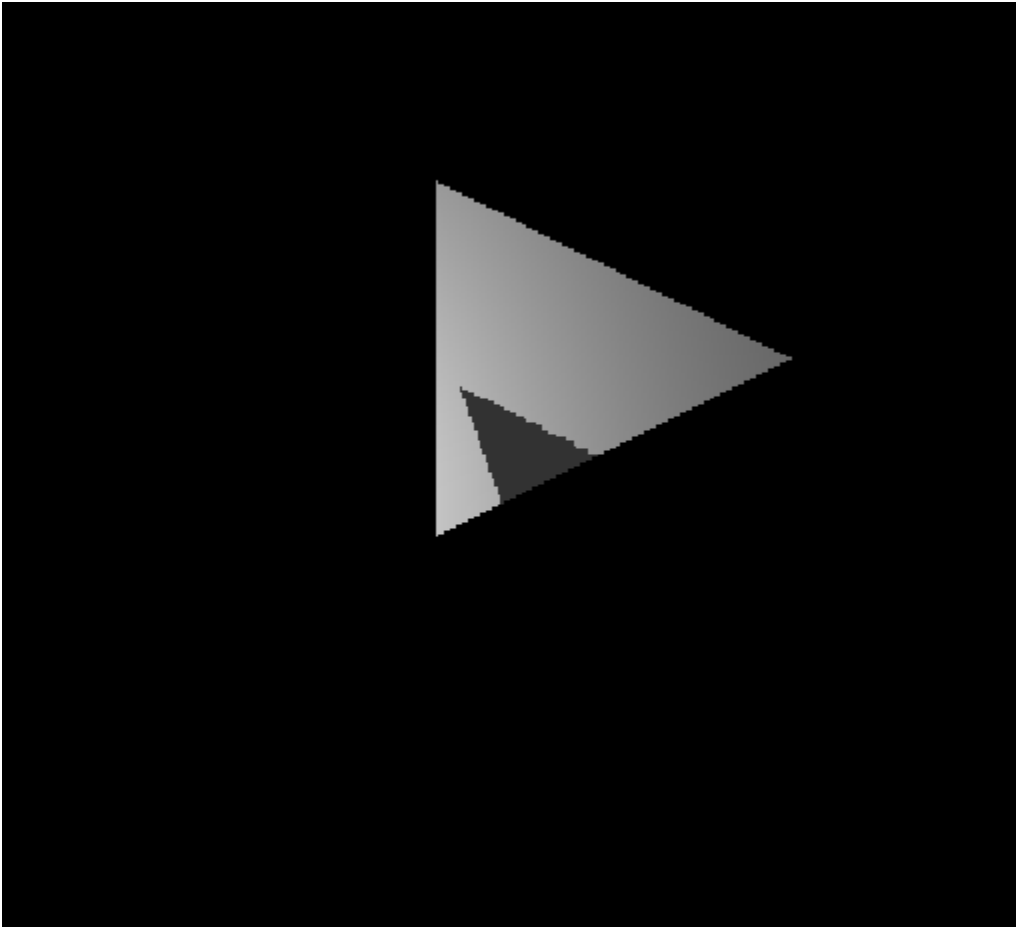
```
figure(9); imshow(Im2);

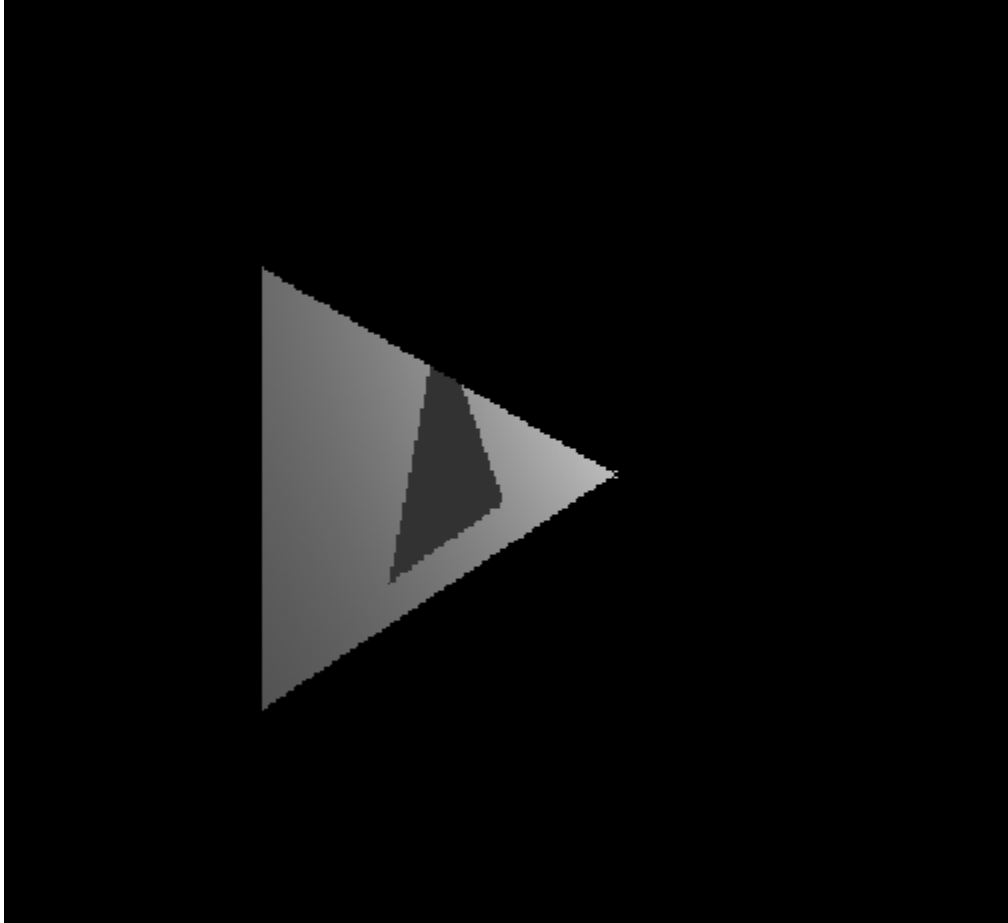
% Z-buffer

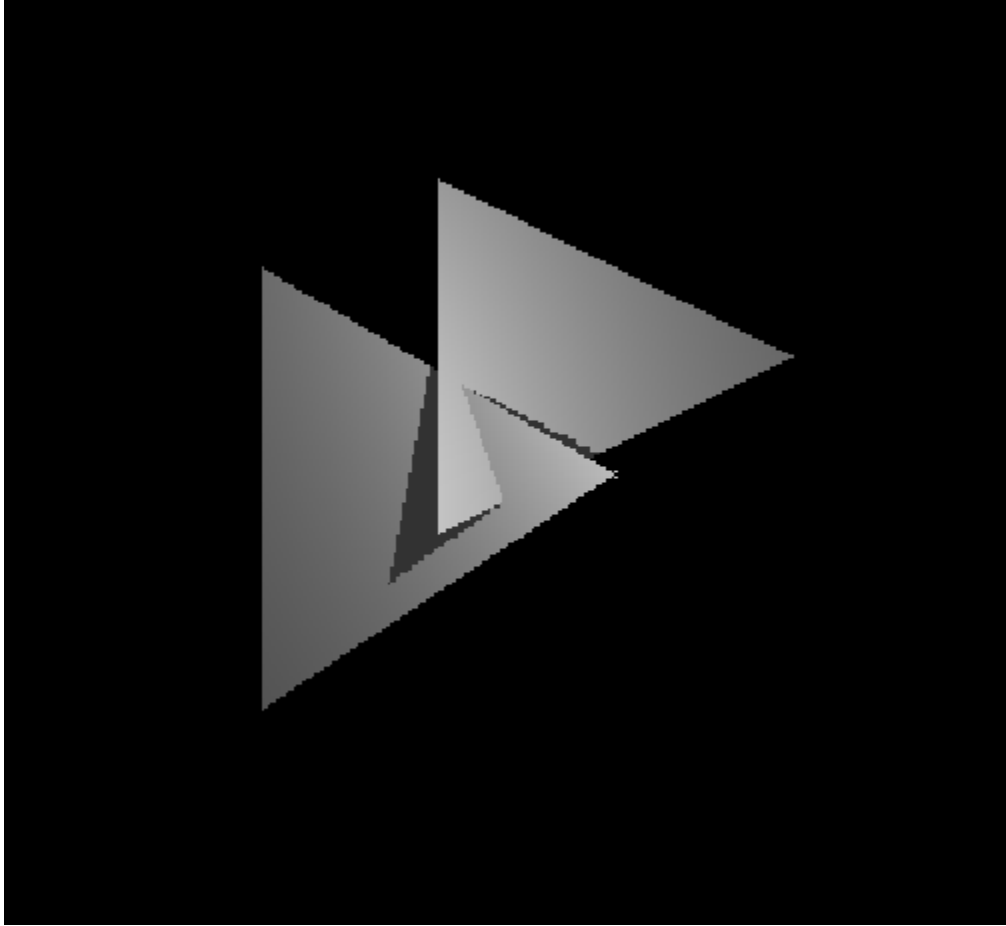
Im = uint8(zeros(Y,X));
z_buffer = inf(Y,X);

for j = 1:Y
    for i = 1:X
        [xx,yy] = pix_to_mat(i, j, X, Y, dx, dy);
        if Ish1(j,i) == 255
            [d,~,~,~] = dist_to_plane([A1, B1, C1, D1], xx, yy, 0, 0, 0, 1);
            if d < z_buffer(j, i)
                z_buffer(j, i) = d;
                Im(j, i, :) = Im1(j, i, :);
            end
        end
        if Ish2(j,i) == 255
            [d,~,~,~] = dist_to_plane([A2, B2, C2, D2], xx, yy, 0, 0, 0, 1);
            if d < z_buffer(j, i)
                z_buffer(j, i) = d;
                Im(j, i, :) = Im2(j, i, :);
            end
        end
    end
end
end

figure(10); imshow(Im);
```







ZADANIE 2

Używając modelu oświetlenia Phong'a, wygeneruj obraz tej sceny.

```
% „Oka obserwatora" w punkcie
```

```
eyea = [-3; -5; 0];
```

```
eyeb = [ 5;  0; 0];
```

```
% m = 5;
```

```
m = 15;
```

```
% m = 50;
```

```
Im1 = uint8(zeros(Y,X));
```

```
Im2 = uint8(zeros(Y,X));
```

```
[Im1, Ish1] = phong(Im1, tri(1), amb, lampa, int, X, Y, dx, dy, e, eyea, m);
```

```
[Im2, Ish2] = phong(Im2, tri(2), amb, lampa, int, X, Y, dx, dy, e, eyea, m);
```

```
figure(11); imshow(Im1);
```

```
figure(12); imshow(Im2);
```

```
% Cienie
```

```

[A1, B1, C1, D1] = plane(tri(1).A, tri(1).B, tri(1).C);
n1 = [A1; B1; C1] / norm([A1; B1; C1]);
if n1(3) > 0
    n1 = -1 * n1;
end

[A2, B2, C2, D2] = plane(tri(2).A, tri(2).B, tri(2).C);
n2 = [A2; B2; C2] / norm([A2; B2; C2]);
if n2(3) > 0
    n2 = -1 * n2;
end

Im1 = shadow(Im1, Ish1, Ish2, A1, B1, C1, D1, A2, B2, C2, D2, X, Y, dx, dy,
lampa, amb);
Im2 = shadow(Im2, Ish2, Ish1, A2, B2, C2, D2, A1, B1, C1, D1, X, Y, dx, dy,
lampa, amb);

figure(13); imshow(Im1);
figure(14); imshow(Im2);

% Z-buffer

Im = uint8(zeros(Y,X));
z_buffer = inf(Y,X);

for j = 1:Y
    for i = 1:X
        [xx,yy] = pix_to_mat(i, j, X, Y, dx, dy);
        if Ish1(j,i) == 255
            [d,~,~,~] = dist_to_plane([A1, B1, C1, D1], xx, yy, 0, 0, 0, 1);
            if d < z_buffer(j, i)
                z_buffer(j, i) = d;
                Im(j, i, :) = Im1(j, i, :);
            end
        end
        if Ish2(j,i) == 255
            [d,~,~,~] = dist_to_plane([A2, B2, C2, D2], xx, yy, 0, 0, 0, 1);
            if d < z_buffer(j, i)
                z_buffer(j, i) = d;
                Im(j, i, :) = Im2(j, i, :);
            end
        end
    end
end
end

figure(15); imshow(Im);

Im1 = uint8(zeros(Y,X));
Im2 = uint8(zeros(Y,X));

```

```

[Im1, Ish1] = phong(Im1, tri(1), amb, lampb, int, X, Y, dx, dy, e, eyeb, m);
[Im2, Ish2] = phong(Im2, tri(2), amb, lampb, int, X, Y, dx, dy, e, eyeb, m);

figure(16); imshow(Im1);
figure(17); imshow(Im2);

% Cienie
[A1, B1, C1, D1] = plane(tri(1).A, tri(1).B, tri(1).C);
n1 = [A1; B1; C1] / norm([A1; B1; C1]);
if n1(3) > 0
    n1 = -1 * n1;
end

[A2, B2, C2, D2] = plane(tri(2).A, tri(2).B, tri(2).C);
n2 = [A2; B2; C2] / norm([A2; B2; C2]);
if n2(3) > 0
    n2 = -1 * n2;
end

Im1 = shadow(Im1, Ish1, Ish2, A1, B1, C1, D1, A2, B2, C2, D2, X, Y, dx, dy,
lampa, amb);
Im2 = shadow(Im2, Ish2, Ish1, A2, B2, C2, D2, A1, B1, C1, D1, X, Y, dx, dy,
lampa, amb);

figure(18); imshow(Im1);
figure(19); imshow(Im2);

% Z-buffer
Im = uint8(zeros(Y,X));
z_buffer = inf(Y,X);

for j = 1:Y
    for i = 1:X
        [xx,yy] = pix_to_mat(i, j, X, Y, dx, dy);
        if Ish1(j,i) == 255
            [d,~,~,~] = dist_to_plane([A1, B1, C1, D1], xx, yy, 0, 0, 0, 1);
            if d < z_buffer(j, i)
                z_buffer(j, i) = d;
                Im(j, i, :) = Im1(j, i, :);
            end
        end
        if Ish2(j,i) == 255
            [d,~,~,~] = dist_to_plane([A2, B2, C2, D2], xx, yy, 0, 0, 0, 1);
            if d < z_buffer(j, i)
                z_buffer(j, i) = d;
                Im(j, i, :) = Im2(j, i, :);
            end
        end
    end
end
end

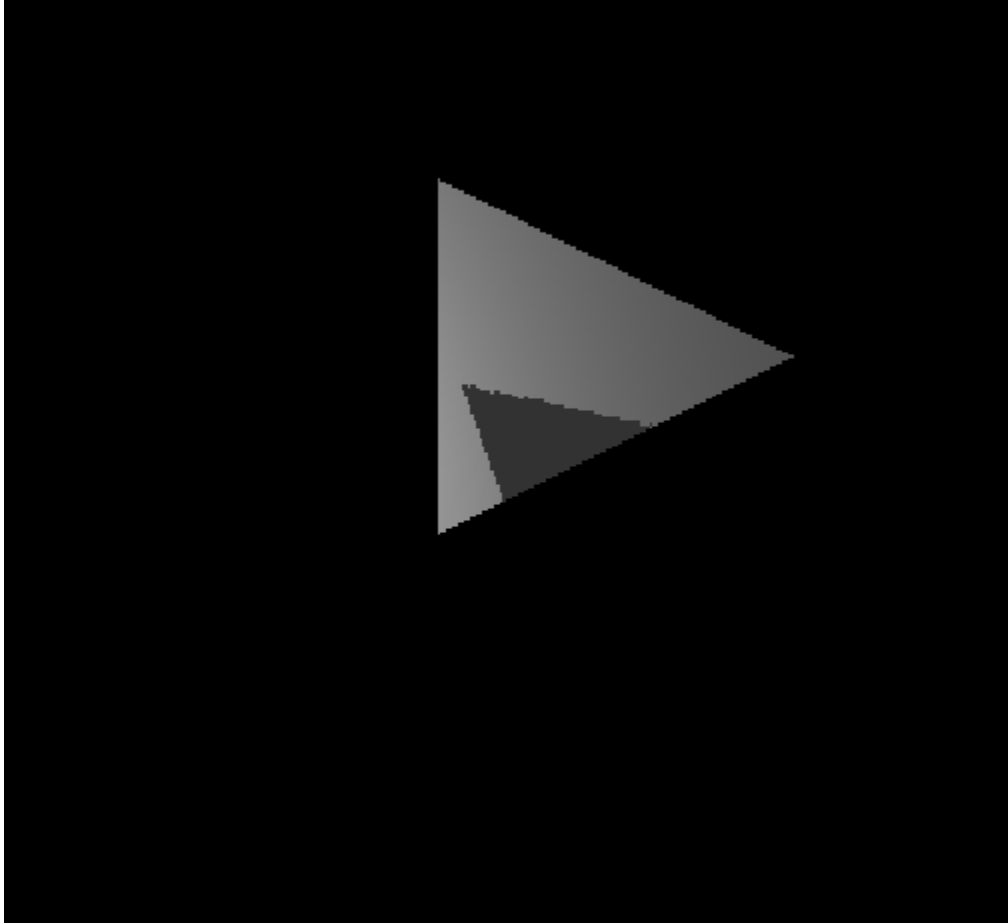
figure(20); imshow(Im);

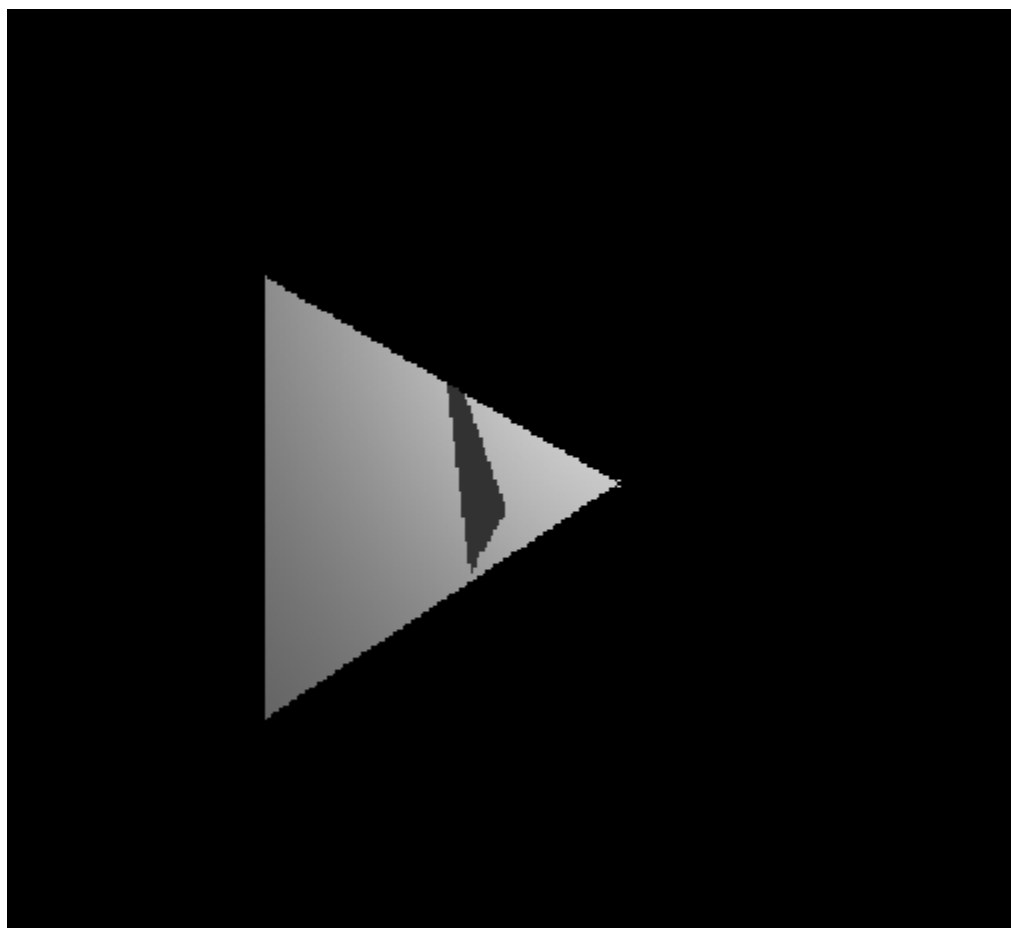
```

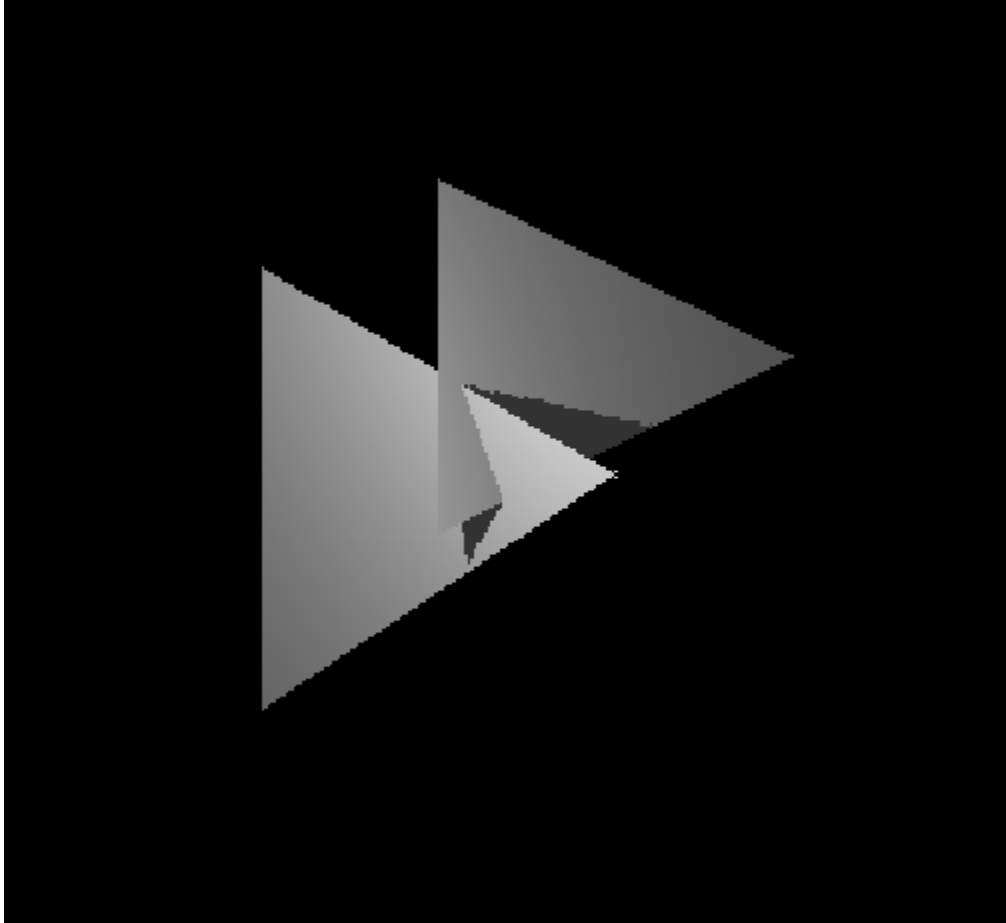
FUNKCJE POMOCNICZE

SHADOW

```
function Im = shadow(Im, Ish1, Ish2, A1, B1, C1, D1, A2, B2, C2, D2, X, Y, dx, dy, lamp, amb)
    for i = 1:X
        for j = 1:Y
            if Ish1(j,i)==255
                % sprawdzanie rzucania cienia
                [xx,yy] = pix_to_mat(i,j,X,Y,dx,dy);
                [~, x, y, z] = dist_to_plane([A1, B1, C1, D1],xx,yy,0,0,0,1);
                l1 = lamp-[x;y;z];
                dist1 = norm(l1);
                [d2, x2, y2, z2] = dist_to_plane([A2, B2, C2,
D2],x,y,z, l1(1), l1(2), l1(3));
                if norm([x2;y2;z2]-lamp)<dist1 && d2<dist1
                    [i2, j2] = mat_to_pix(x2,y2,X,Y,dx,dy);
                    if i2>0 && j2>0 && i2<X+1 && j2<Y+1 && Ish2(j2,i2)==255
                        Im(j,i) = amb;
                    end
                end
            end
        end
    end
end
```







LAMBERT

```
function [Im, Ish] = lambert(Im, triangle, amb, lamp, int, X, Y, dx, dy)
    Ish = uint8(zeros(Y,X)); % pomocniczy obraz zawierający biały trójkąt

    [i1,j1] = mat_to_pix(triangle.A(1), triangle.A(2), X, Y, dx, dy);
    [i2,j2] = mat_to_pix(triangle.B(1), triangle.B(2), X, Y, dx, dy);
    [i3,j3] = mat_to_pix(triangle.C(1), triangle.C(2), X, Y, dx, dy);

    Ish = line(Ish, j1, i1, j2, i2, dx, dy, 255);
    Ish = line(Ish, j2, i2, j3, i3, dx, dy, 255);
    Ish = line(Ish, j3, i3, j1, i1, dx, dy, 255);

    Ish = floodfill(Ish, round((j1+j2+j3)/3), round((i1+i2+i3)/3), 0, 255); %
    wypełnienie trójkąta białym kolorem

    [A, B, C, D] = plane(triangle.A, triangle.B, triangle.C); % Wyznaczenie
    parametrów płaszczyzny trójkąta
    n = [A; B; C] / norm([A; B; C]); % Normalizacja wektora normalnego
    if n(3) > 0
        n = -1*n;
    end % Obrócenie normalnej, jeśli jest skierowana w dół
```

```

    for i = 1:X
        for j = 1:Y
            if Ish(j,i) == 255 % Sprawdzenie, czy piksel jest wewnątrz
                trójkąta
                    [xx,yy] = pix_to_mat(i, j, X, Y, dx, dy); % Zamiana
                    współrzędnych pikselowych na matematyczne
                    [~, x, y, z] = dist_to_plane([A,B,C,D], xx, yy, 0, 0, 0, 1);
                    % Wyznaczenie punktu na płaszczyźnie trójkąta
                    l = lamp - [x; y; z]; % Wektor od punktu na trójkącie do lampy
                    dist = norm(l); % Odległość od lampy
                    l = l / dist; % Normalizacja wektora l
                    cs = n'* l;
                    if cs < 0
                        cs = 0;
                    end % Obliczenie cosinusa kąta między normalną a wektorem do
                    lampy
                    temp = amb + int * cs / (1 + 0.001 * dist^2); % Obliczenie
                    natężenia oświetlenia z uwzględnieniem tłumienia
                    if temp > 255
                        temp = 255;
                    end % Ograniczenie natężenia do 255
                    Im(j, i) = temp; % Ustawienie natężenia dla pikseli wewnątrz
                    trójkąta
                end
            end
        end
    end
end

```

PHONG

```

function [Im, Ish] = phong(Im, triangle, amb, lamp, int, X, Y, dx, dy, e,
eye, m)
    Ish = uint8(zeros(Y,X)); % pomocniczy obraz zawierający biały trójkąt

    [i1,j1] = mat_to_pix(triangle.A(1), triangle.A(2), X, Y, dx, dy);
    [i2,j2] = mat_to_pix(triangle.B(1), triangle.B(2), X, Y, dx, dy);
    [i3,j3] = mat_to_pix(triangle.C(1), triangle.C(2), X, Y, dx, dy);

    Ish = line(Ish, j1, i1, j2, i2, dx, dy, 255);
    Ish = line(Ish, j2, i2, j3, i3, dx, dy, 255);
    Ish = line(Ish, j3, i3, j1, i1, dx, dy, 255);

    Ish = floodfill(Ish, round((j1+j2+j3)/3), round((i1+i2+i3)/3), 0, 255); %
    wypełnienie trójkąta białym kolorem

    [A, B, C, D] = plane(triangle.A, triangle.B, triangle.C); % Wyznaczenie
    parametrów płaszczyzny trójkąta
    n = [A; B; C] / norm([A; B; C]); % Normalizacja wektora normalnego
    if n(3) > 0
        n = -1*n;
    end % Obrócenie normalnej, jeśli jest skierowana w dół

```

```

    for i = 1:X
        for j = 1:Y
            if Ish(j,i) == 255 % Sprawdzenie, czy piksel jest wewnątrz
                trójkąta
                    [xx,yy] = pix_to_mat(i, j, X, Y, dx, dy); % Zamiana
                    współrzędnych pikselowych na matematyczne
                    [~, x, y, z] = dist_to_plane([A,B,C,D], xx, yy, 0, 0, 0, 1);
                    % Wyznaczenie punktu na płaszczyźnie trójkąta
                    l = lamp - [x; y; z]; % Wektor od punktu na trójkącie do lampy
                    dist = norm(l); % Odległość od lampy
                    l = l / dist; % Normalizacja wektora l
                    cs = n'* l;
                    if cs < 0
                        cs = 0;
                    end % Obliczenie cosinusa kąta między normalną a wektorem do
                    lampy

                    rp = 2 * (n'* l) * n - l; % Wektor odbicia
                    rp = rp / norm(rp); % Normalizacja wektora odbicia
                    vp = eye - [x; y; z]; % Wektor od punktu na trójkącie do oka
                    vp = vp / norm(vp); % Normalizacja wektora do oka
                    cs2 = rp'* vp;
                    if cs2 < 0
                        cs2 = 0;
                    end % Obliczenie cosinusa kąta między wektorem odbicia a
                    wektorem do oka

                    temp = amb + int * cs * e * (1 + (cs2^m)) / (1 + 0.001 *
                    dist^2); % Obliczenie natężenia oświetlenia z uwzględnieniem tłumienia
                    if temp > 255
                        temp = 255;
                    end % Ograniczenie natężenia do 255 z uwzględnieniem tłumienia
                    Im(j, i) = temp; % Ustawienie natężenia dla pikseli wewnątrz
                    trójkąta
                end
            end
        end
    end
end

```

Funkcja do zamiany współrzędnych matematycznych na pikselowe i odwrotnie

x, y - współrzędne M, N - szerokość i wysokość obrazu dx, dy - rozmiar pojedynczego piksela

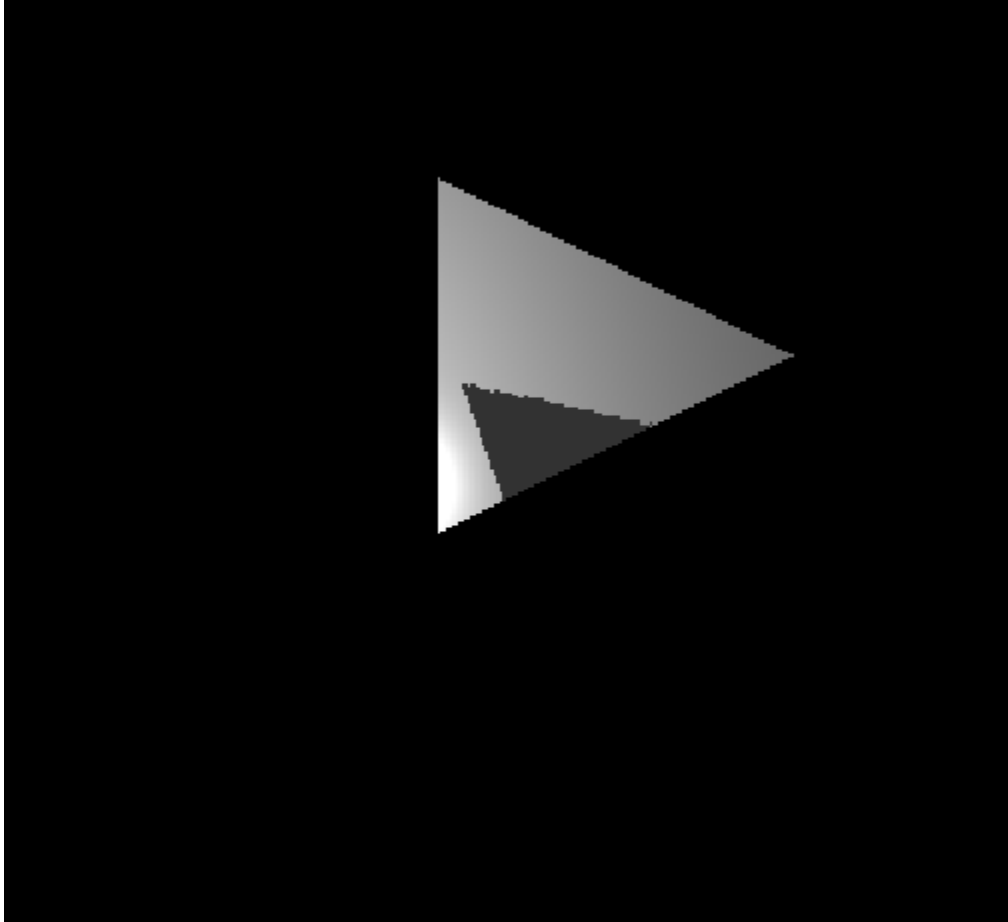
```

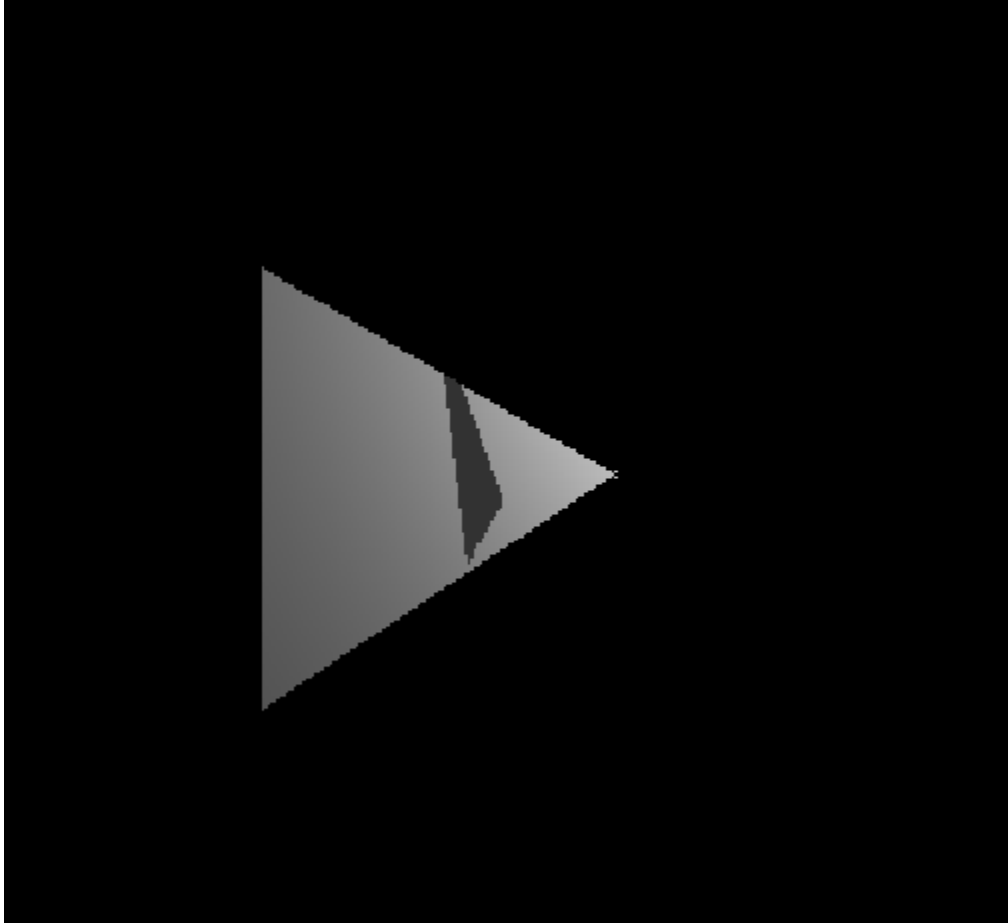
function [i,j] = mat_to_pix(x, y, M, N, dx, dy)
    T = [1/dx, 0, M/2;
         0, -1/dy, N/2;
         0, 0, 1];
    temp = [x; y; 1];
    res = T * temp;
    i = round(res(1)/res(3));
    j = round(res(2)/res(3));

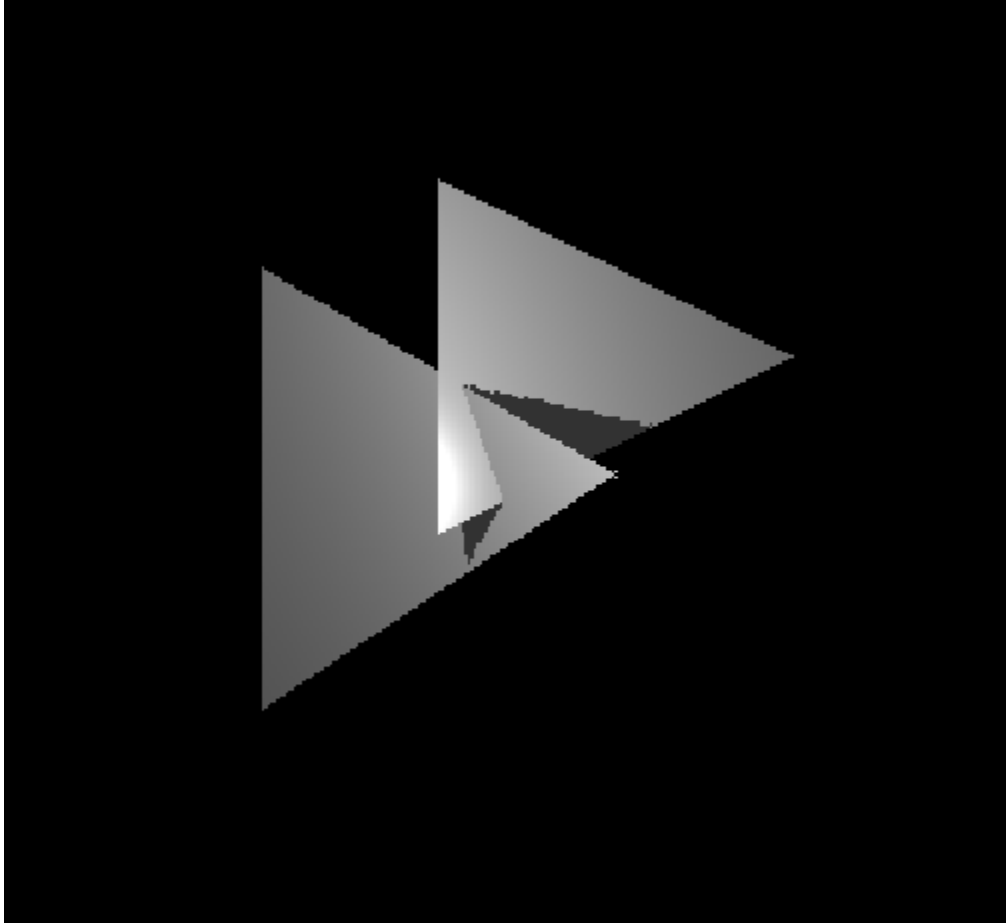
```

end

```
function [i,j] = pix_to_mat(x, y, M, N, dx, dy)
    T = [dx, 0, -M*dx/2;
         0, -dy, N*dy/2;
         0, 0, 1];
    res = T * [x; y; 1];
    i = res(1)/res(3);
    j = res(2)/res(3);
end
```







Funkcja rysuje odcinek na obrazie monochromatycznym.

```
function Im = line(Im, ya, xa, yb, xb, dx, dy, color)
    % Im - obraz
    % (xa, ya) - punkt początkowy
    % (xb, yb) - punkt końcowy
    % color - kolor linii

    [Y, X] = size(Im);

    if (abs(yb-ya) > abs(xb-xa))           % stroma linia
        % zamiana współrzędnych
        x0 = ya;
        y0 = xa;
        x1 = yb;
        y1 = xb;

        % zamiana rozmiarów X i Y
        temp = X; X = Y; Y = temp;

        zamiana = 1;
    end
```

```

else
    x0 = xa;
    y0 = ya;
    x1 = xb;
    y1 = yb;

    zamiana = 0;
end

% zamiana poczatk i konca linii, zeby zaczynac od lewej
if(x0 > x1)
    temp1 = x0; x0 = x1; x1 = temp1;
    temp2 = y0; y0 = y1; y1 = temp2;
end

dx = abs(x1 - x0) ;           % odleglosc x
dy = abs(y1 - y0);           % odleglosc y
sy = sign(y1 - y0);           % znak przyrostu w kierunku y

x = x0; y = y0;               % inicjalizacja
if x > 0 && x <= X && y > 0 && y <= Y
    if (zamiana == 0)
        Im(y,x) = color;      % rysowanie punktu
    else
        Im(x,y) = color;
    end
end

error = 2*dy - dx ;           % inicjalizacja bledu
for i = 0:dx-1

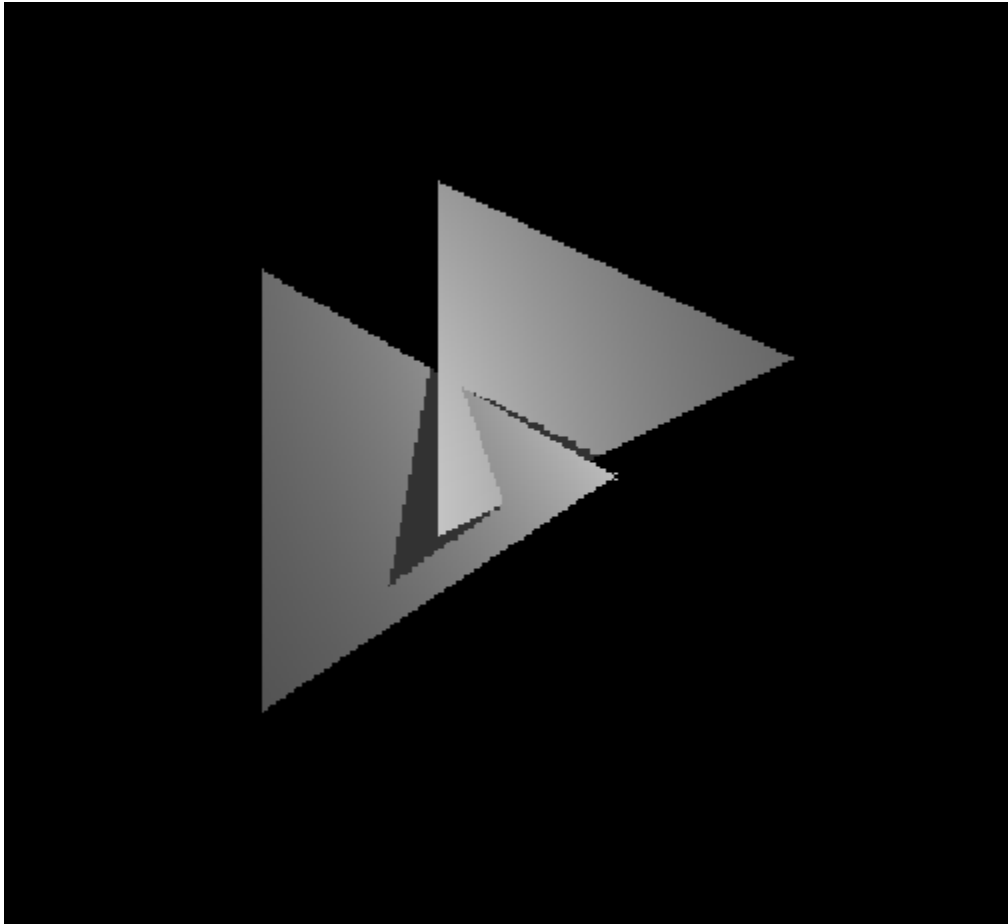
    error = error + 2*dy;      % modyfikacja bledu
    if (error > 0)
        y = y + sy;
        error = error - 2*dx;
    end

    x = x + 1;                 % zwiekszamy x
    if x > 0 && x <= X && y > 0 && y <= Y
        if (zamiana == 0)
            Im(y,x) = color;    % rysowanie punktu
        else
            Im(x,y) = color;
        end
    end
end
end
end

```

Funkcja do wyznaczania parametrow płaszczyzny $Ax+By+Cz+D=0$ przechodzącej przez trzy punkty p1,p2,p3

```
function [A, B, C, D] = plane(p1,p2,p3)
    A = det([p1(2),p1(3),1;
             p2(2),p2(3),1;
             p3(2),p3(3),1]);
    B = -det([p1(1),p1(3),1;
              p2(1),p2(3),1;
              p3(1),p3(3),1]);
    C = det([p1(1),p1(2),1;
             p2(1),p2(2),1;
             p3(1),p3(2),1]);
    D = -det([p1(1),p1(2),p1(3);
              p2(1),p2(2),p2(3);
              p3(1),p3(2),p3(3)]);
end
```



Funkcja do wyznaczania odleglosci punktu $[x,y,z]$ od plaszczyzny o rownaniu $Ax+By+Cz+D=0$

odleglosc mierzona jest wzdluz prostej o wektorze kierunkowym $[l,m,n]$ zwraca rowniez punkt przeciecia z plaszczyzna

```
function [d,x1,y1,z1] = dist_to_plane(plane, x, y, z, l, m, n)
    ro = (plane(1)*x+plane(2)*y+plane(3)*z+plane(4))/
(plane(1)*l+plane(2)*m+plane(3)*n);
    x1 = x-l*ro; y1 = y-m*ro; z1 = z-n*ro;
    d = sqrt((x-x1)^2+(y-y1)^2+(z-z1)^2);
end
```

Funkcja wypelnia zamkniety obszar obrazu metoda floodfill.

```
function Im = floodfill(Im, y, x, T, color)
    % Im - obraz
    % (y,x) - punkt startowy
    % T - jasnosc do zamiany
    % color - nowy kolor

    [Y, X] = size(Im);

    % sprawdzenie zakresu
    if y < 1 || y > Y || x < 1 || x > X
        error('Punkt startowy (y,x) jest poza obrazem.');
```

end

```
    % sprawdzenie jasnosci piksela startowego
    if Im(y,x) ~= T
        %disp('Piksel startowy ma zla jasnosc.');
```

return;

```
    % w naszym kodzie oznacza to że trafiliśmy na krawędź - a więc
    trójkąt jest obrucony do nas bokiem.
    end

    % jesli nowa jasnosc taka sama jak stara, nic nie trzeba robic
    if T == color
        return;
    end

    % kolejka pikseli do odwiedzenia
    Q = zeros(Y*X, 2);
    head = 1;
    tail = 1;

    Q(tail,:) = [y, x];
```

```
Im(y,x) = color;

while head <= tail
    cy = Q(head,1);
    cx = Q(head,2);
    head = head + 1;

    % 4-sasiedztwo: gora, dol, lewo, prawo

    % gora
    ny = cy - 1;
    nx = cx;
    if ny >= 1 && Im(ny,nx) == T
        tail = tail + 1;
        Q(tail,:) = [ny, nx];
        Im(ny,nx) = color;
    end

    % dol
    ny = cy + 1;
    nx = cx;
    if ny <= Y && Im(ny,nx) == T
        tail = tail + 1;
        Q(tail,:) = [ny, nx];
        Im(ny,nx) = color;
    end

    % lewo
    ny = cy;
    nx = cx - 1;
    if nx >= 1 && Im(ny,nx) == T
        tail = tail + 1;
        Q(tail,:) = [ny, nx];
        Im(ny,nx) = color;
    end

    % prawo
    ny = cy;
    nx = cx + 1;
    if nx <= X && Im(ny,nx) == T
        tail = tail + 1;
        Q(tail,:) = [ny, nx];
        Im(ny,nx) = color;
    end
end
end
```

Published with MATLAB® R2026a